Subs - O(confia)

Contents

1 2013 1
- 1.1 A . . . 1
- 1.2 B . . . 1
- 1.3 C . . . 2
- 1.4 D . . . 3
- 1.5 E . . . 4
- 1.6 F . . . 4
- 1.7 G . . . 4
- 1.8 H . . . 5
- 1.9 I . . . 6
- 1.10 J . . . 6

2 2018 7
- 2.1 A . . . 7
- 2.2 B . . . 7
- 2.3 C . . . 8
- 2.4 D . . . 8
- 2.5 E . . . 8
- 2.6 F . . . 9
- 2.7 G . . . 9
- 2.8 I . . . 10
- 2.9 J . . . 11
- 2.10 L . . . 11

3 2019 13
- 3.1 A . . . 13
- 3.2 B . . . 13
- 3.3 D . . . 13
- 3.4 G . . . 14
- 3.5 H . . . 15
- 3.6 I . . . 15
- 3.7 J . . . 16
- 3.8 K . . . 16
- 3.9 L . . . 17
- 3.10 M . . . 17

4 2020 17
- 4.1 A . . . 17
- 4.2 B . . . 18
- 4.3 E . . . 18
- 4.4 F . . . 19
- 4.5 G . . . 19
- 4.6 H . . . 19
- 4.7 I . . . 20
- 4.8 L . . . 20
- 4.9 N . . . 21

5 2021 21
- 5.1 A . . . 21
- 5.2 B . . . 22
- 5.3 C . . . 23
- 5.4 D (wrong) . . . 23
- 5.5 E . . . 24
- 5.6 G . . . 25
- 5.7 H . . . 25
- 5.8 K . . . 25
- 5.9 L . . . 26
- 5.10 M . . . 26
- 5.11 N . . . 27

6 2022 28
- 6.1 A . . . 28
- 6.2 B . . . 28
- 6.3 C . . . 28
- 6.4 D . . . 29
- 6.5 E . . . 29
- 6.6 F . . . 29
- 6.7 G . . . 30
- 6.8 H . . . 30
- 6.9 I . . . 31
- 6.10 J . . . 31
- 6.11 K . . . 32
- 6.12 L . . . 33
- 6.13 M . . . 33
- 6.14 N . . . 34

1 2013

1.1 A

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int A, B, C;
    cin >> A >> B >> C;

    if (A != B && A != C) cout << "A\n";
    else if (B != A && B != C) cout << "B\n";
    else if (C != A && C != B) cout << "C\n";
    else cout << "*\n";
}
```

1.2 B

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

#define BEGIN 0
#define BALLOON 1
#define END 2

const long double EPS = 1e-9;

int sign(long double x) {
    return abs(x) < EPS ? 0 : (x > 0 ? 1 : -1);
}

template<typename T>
struct Point {
    T x, y;

    Point(): x(0), y(0) {}
    Point(T x, T y): x(x), y(y) {}
};

struct Event {
    int x, type, idx;

    Event(int x, int type, int idx): x(x), type(type), idx(idx) {}

```

```cpp
    bool operator <(const Event &other) const {
        if (x == other.x)
            return type < other.type;

        return x < other.x;
    }
};

struct Segment {
    int idx;
    Point<llong> P, Q;

    Segment(int idx, Point<llong> P, Point<llong> Q): idx(idx) {
        if (P.x > Q.x) swap(P, Q);
        this->P = P;
        this->Q = Q;
    }

    long double get_y(llong x) const {
        if (P.x == Q.x) return P.y;

        long double m = 1.0L * (Q.y - P.y) / (Q.x - P.x);
        return P.y + m * (x - P.x);
    }

    bool operator <(const Segment &other) const {
        llong x = max(P.x, other.P.x);
        int y1 = get_y(x), y2 = other.get_y(x);

        if (!sign(y1 - y2))
            return idx < other.idx;

        return sign(y1 - y2) < 0;
    }
};

void get_exit(vector<vector<int>> &forest, vector<int> &exit, vector<int> &slast, int node, int idx, int sidx = -1) {
    exit[node] = idx;
    slast[node] = sidx;

    for (int child: forest[node])
        get_exit(forest, exit, slast, child, idx, node == idx ? child : sidx);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, C;
    while (cin >> N >> C) {
        vector<Event> events;
        vector<Segment> segments;

        for (int i = 0; i < N; i++) {
            int X1, Y1, X2, Y2;
            cin >> X1 >> Y1 >> X2 >> Y2;

            events.emplace_back(min(X1, X2), BEGIN, i);
            events.emplace_back(max(X1, X2), END, i);

            Point<llong> P(X1, Y1), Q(X2, Y2);
            segments.emplace_back(i, P, Q);
        }

        vector<int> balloons(C);
        for (int i = 0; i < C; i++) {
            cin >> balloons[i];
            events.emplace_back(balloons[i], BALLOON, i);
        }

        sort(events.begin(), events.end());

        set<Segment> active;
        vector<int> parent(N, -1), first(C, -1);

        for (auto [x, type, idx]: events) {
            if (type == BEGIN) {
                active.insert(segments[idx]);
                if (segments[idx].P.y > segments[idx].Q.y) {
                    auto it = active.upper_bound(segments[idx]);
                    if (it != active.end()) parent[idx] = it->idx;
                }
            } else if (type == END) {
                if (segments[idx].P.y < segments[idx].Q.y) {
                    auto it = active.upper_bound(segments[idx]);
                    if (it != active.end()) parent[idx] = it->idx;
                }

                active.erase(segments[idx]);
            } else {
                auto it = active.begin();
                if (it != active.end())
                    first[idx] = it->idx;
            }
        }

        vector<vector<int>> forest(N, vector<int>());
        for (int i = 0; i < N; i++)
            if (parent[i] != -1)
                forest[parent[i]].push_back(i);

        vector<int> exit(N, -1), slast(N);
        for (int i = 0; i < N; i++)
            if (exit[i] == -1)
                get_exit(forest, exit, slast, i, i);

        for (int i = 0; i < C; i++) {
            if (first[i] == -1) cout << balloons[i] << "\n";
            else {
                int last = exit[first[i]];
                if (segments[last].P.y > segments[last].Q.y) cout << segments[last].P.x << "\n";
                else if (segments[last].P.y < segments[last].Q.y) cout << segments[last].Q.x << "\n";
                else if (first[i] == last) cout << balloons[i] << " " << segments[last].P.y << "\n";
                else {
                    int idx = slast[first[i]];
                    assert(idx != -1);

                    int x = segments[idx].P.y > segments[idx].Q.y ? segments[idx].P.x : segments[idx].Q.x;
                    cout << x << " " << segments[last].P.y << "\n";
                }
            }
        }
    }
}
```

1.3 C

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int INF = 1e6 + 5;

vector<int> K, id;
vector<vector<int>> g;
```

```cpp
vector<int> memo;

int youngest(int v) {
    int &ans = memo[v];
    if (~ans) return ans;

    ans = INF;
    for (int u: g[v]) {
        int manager = id[u];
        ans = min(ans, K[manager]);
        ans = min(ans, youngest(u));
    }

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M, I;
    cin >> N >> M >> I;

    K.resize(N);
    id.resize(N);

    vector<int> pos(N);
    for (int i = 0; i < N; i++) {
        cin >> K[i];
        id[i] = pos[i] = i;
    }

    g.assign(N, vector<int>());
    for (int i = 0; i < M; i++) {
        int X, Y;
        cin >> X >> Y;
        g[Y - 1].push_back(X - 1);
    }

    memo.assign(N + 5, -1);
    while (I--) {
        char op; cin >> op;
        if (op == 'T') {
            int A, B;
            cin >> A >> B;

            swap(id[pos[A - 1]], id[pos[B - 1]]);
            swap(pos[A - 1], pos[B - 1]);
            memo.assign(N + 5, -1);
        } else {
            int E; cin >> E;

            int ans = youngest(pos[E - 1]);
            if (ans >= INF) cout << "*\n";
            else cout << ans << "\n";
        }
    }
}
```

1.4 D

```cpp
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int N, M;
vector<int> input, output;

bool possible() {
    if ((int) input.size() < M) return false;
    if ((int) input.size() == M) {
        bool equal = true;
        for (int i = 0; i < M && equal; i++)
            if (input[i] != output[i])
                equal = false;

        if (equal) return true;

        equal = true;
        for (int i = 0, j = M - 1; i < M; i++, j--)
            if (input[j] != output[i])
                equal = false;

        return equal;
    }

    if (*min_element(input.begin(), input.end()) > *max_element(output.
        begin(), output.end()))
        return false;

    for (int i = 1; i < (int) input.size(); i++) {
        vector<int> first;
        for (int j = 0; j < i; j++)
            first.push_back(j);

        vector<int> last;
        for (int j = (int) input.size() - 1; j >= i; j--)
            last.push_back(j);

        int size_first = first.size();
        int size_last = last.size();

        vector<int> aux = input;
        input.clear();

        int l = 0, r = 0;
        while (size_first - l > size_last)
            input.push_back(aux[first[l++]]);

        while (size_last - r > size_first)
            input.push_back(aux[last[r++]]);

        while (l < size_first)
            input.push_back(aux[first[l++]] + aux[last[r++]]);

        if (possible())
            return true;

        input = aux;
    }

    return false;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> N;

    input.resize(N);
    for (int i = 0; i < N; i++)
        cin >> input[i];

    cin >> M;

    output.resize(M);
    for (int i = 0; i < M; i++)
        cin >> output[i];
```

```
    cout << (possible() ? "S" : "N") << "\n";
}
```

1.5 E

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, R;
    cin >> N >> R;

    vector<bool> returned(N);
    for (int i = 0; i < R; i++) {
        int id; cin >> id;
        returned[id - 1] = true;
    }

    vector<int> ans;
    for (int i = 0; i < N; i++)
        if (!returned[i])
            ans.push_back(i + 1);

    int size = ans.size();
    if (!size) cout << "*\n";
    else {
        for (int i = 0; i < size; i++) {
            if (i) cout << " ";
            cout << ans[i];
        }

        cout << "\n";
    }
}
```

1.6 F

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    int sum = 0;
    vector<int> X(2 * N);

    for (int i = 0; i < N; i++) {
        cin >> X[i];
        X[N + i] = X[i];
        sum += X[i];
    }

    if (sum % 3) {
        cout << "0\n";
        return 0;
    }

    int rsum = 0;
    map<int, int> rcnt;

    for (int i = 2 * N - 1; i >= 0; i--) {
        rsum += X[i];
        rcnt[rsum]++;
    }

    int lsum = 0;
    map<int, int> lcnt;
    lcnt[0] = rcnt[0] = 1;

    int ans = 0;
    for (int i = 0; i < N; i++) {
        lsum += X[i];
        rsum -= X[i];

        lcnt[lsum]++;
        rcnt[rsum]--;

        if (lcnt[lsum - sum / 3] && rcnt[rsum - sum / 3])
            ans++;
    }

    ans /= 2;
    cout << ans << "\n";
}
```

1.7 G

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

struct FenwickTree {
private:
    int n;
    vector<int> tree;

public:
    FenwickTree(int n) {
        this->n = n + 1;
        tree.assign(n + 1, 0);
    }

    void clear() {
        tree.assign(n + 1, 0);
    }

    void add(int i, int v) {
        if (!i) tree[0] += v;
        else {
            while (i < n) {
                tree[i] += v;
                i += LSO(i);
            }
        }
    }

    int sum(int i, int j) {
        return sum(j) - sum(i - 1);
    }

private:
    int LSO(int i) {
        return i & -i;
    }
```

```cpp

    int sum(int i) {
        if (i < 0) return 0;

        int ans = tree[0];
        while (i) {
            ans += tree[i];
            i -= LSO(i);
        }

        return ans;
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int L, C;
    cin >> L >> C;

    vector<vector<int>> X(L, vector<int>(C));
    for (int i = 0; i < L; i++)
        for (int j = 0; j < C; j++)
            cin >> X[i][j];

    /* int ans = 0;
    FenwickTree tree(L * C);

    for (int i = 0; i < C; i++) {
        ans += tree.sum(X[0][i] + 1, L * C);
        tree.add(X[0][i], 1);
    } */

    int ans = 0;
    for (int i = 0; i < C; i++) {
        int idx = i;
        for (int j = i + 1; j < C; j++)
            if (X[0][j] < X[0][idx])
                idx = j;

        for (int j = 0; j < L; j++)
            swap(X[j][i], X[j][idx]);

        ans += i != idx;
    }

    for (int i = 0; i < L; i++) {
        int idx = i;
        for (int j = i + 1; j < L; j++)
            if (X[j][0] < X[idx][0])
                idx = j;

        for (int j = 0; j < C; j++)
            swap(X[i][j], X[idx][j]);

        ans += i != idx;
    }

    int cnt = 1;
    bool possible = true;

    for (int i = 0; i < L && possible; i++)
        for (int j = 0; j < C && possible; j++)
            if (X[i][j] != cnt++) possible = false;

    if (!possible) cout << "*\n";
    else cout << ans << "\n";
}
```

1.8 H

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 2;
const int MOD = 1e6;

template<typename T>
struct Matrix {
    int rows, cols;
    T m[MAX][MAX];

    Matrix(int rows, int cols): rows(rows), cols(cols) {
        memset(m, 0, sizeof m);
    }

    Matrix operator *(Matrix &other) {
        Matrix ans(rows, other.cols);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < other.cols; j++)
                for (int k = 0; k < cols; k++) {
                    ans[i][j] += m[i][k] * other[k][j];
                    ans[i][j] %= MOD;
                }

        return ans;
    }

    T* operator [](int row) {
        return m[row];
    }
};

template<typename T>
Matrix<T> exp(Matrix<T> &A, llong n) {
    if (n == 1) return A;

    Matrix<T> ans = exp(A, n >> 1LL);
    ans = ans * ans;

    if (!(n & 1LL)) return ans;
    return ans * A;
}

string pad(int x) {
    string ans = "";
    while (x) {
        char c = '0' + (x % 10);
        ans += c;
        x /= 10;
    }

    while ((int) ans.size() < 6)
        ans += '0';

    reverse(ans.begin(), ans.end());
    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    llong N, K, L;
    cin >> N >> K >> L;

    Matrix<llong> A(2, 2);
    A[0][0] = K % MOD;
```

```
    A[0][1] = L % MOD;
    A[1][0] = 1;

    Matrix<llong> init(2, 1);
    init[0][0] = K % MOD;
    init[1][0] = 1;

    int ans = K % MOD;
    if (N > 5LL) {
        N /= 5LL;
        Matrix<llong> aux = exp(A, N - 1LL) * init;
        ans = aux[0][0];
    }

    cout << pad(ans) << "\n";
}
```

1.9 I

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int N, C, T1, T2;
vector<bool> hole;
vector<int> memo;

int length(int dist = 0) {
    if (dist >= C) return 0;

    int &ans = memo[dist];
    if (~ans) return ans;

    ans = T1 + length(dist + T1 + 1);
    ans = min(ans, T2 + length(dist + T2 + 1));
    if (!hole[dist]) ans = min(ans, length(dist + 1));

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> N >> C >> T1 >> T2;

    hole.assign(C, false);
    for (int i = 0; i < N; i++) {
        int F; cin >> F;
        hole[F] = true;
    }

    memo.assign(C + 5, -1);

    int ans = length();
    cout << ans << "\n";
}
```

1.10 J

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

struct PersistentDSU {
private:
    int n;
    vector<int> ds, sz, tm;

public:
    PersistentDSU(int n) {
        this->n = n;
        sz.assign(n, 1);
        tm.assign(n, 0);

        ds.resize(n);
        iota(ds.begin(), ds.end(), 0);
    }

    int find(int i, int t = INT_MAX) {
        if (ds[i] == i || tm[i] > t) return i;
        return find(ds[i], t);
    }

    void join(int i, int j, int t) {
        i = find(i); j = find(j);
        if (i == j) return;

        if (sz[i] > sz[j]) swap(i, j);
        sz[j] += sz[i];
        ds[i] = j;
        tm[i] = t;
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M, S;
    cin >> N >> M >> S;

    vector<tuple<int, int, int>> edges(M);
    for (int i = 0; i < M; i++) {
        int A, B, W;
        cin >> A >> B >> W;
        edges[i] = make_tuple(W, A - 1, B - 1);
    }

    sort(edges.rbegin(), edges.rend());

    PersistentDSU tree(N);
    for (int i = 0; i < M; i++) {
        auto [w, u, v] = edges[i];
        tree.join(u, v, i);
    }

    while (S--) {
        int L, H;
        cin >> L >> H;

        int l = 0, r = M - 1, ans = -1;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (tree.find(L - 1, mid) != tree.find(H - 1, mid)) l = mid + 1;
            else r = mid - 1, ans = mid;
        }

        assert(ans != -1);
        ans = get<0>(edges[ans]);

        cout << ans << "\n";
    }
}
```

2 2018

2.1 A

```cpp
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, C, T;
    cin >> N >> C >> T;

    vector<int> P(N);
    int sum = 0, mx = INT_MIN;

    for (int i = 0; i < N; i++) {
        cin >> P[i];
        sum += P[i];
        mx = max(mx, P[i]);
    }

    int l = mx, r = sum, ans;
    while (l <= r) {
        int mid = (l + r) / 2;

        int cnt = 1, curr_sum = 0;
        for (int i = 0; i < N; i++) {
            if (curr_sum + P[i] <= mid) curr_sum += P[i];
            else {
                cnt++;
                curr_sum = P[i];
            }
        }

        if (cnt > C) l = mid + 1;
        else r = mid - 1, ans = mid;
    }

    ans = (ans + T - 1) / T;
    cout << ans << "\n";
}
```

2.2 B

```cpp
#include "bits/stdc++.h"
using namespace std;

const int MAXN = 105;

int vis[3*MAXN];
int grid[MAXN][MAXN] = {};

void print(){
    for(int i = 0; i <= 10; i++){
        for(int j = 0; j <= 10; j++){
            cout << grid[i][j] << ' ';
        }
        cout << '\n';
    }
}

void calc(){
    for(int i = 0; i <= 100; i++){
        grid[0][i] = grid[i][0] = grid[i][i] = -1;  //posicoes terminais
    }

    for(int i = 0; i <= 100; i++){
        for(int j = 0; j <= 100; j++){

            memset(vis, 0, sizeof(vis));

            if(grid[i][j] == -1) continue;

            //cima
            for(int di = 1; di <= i; di++){
                if(grid[i-di][j] != -1) vis[grid[i-di][j]] = 1;
                //grid[i][j] = max(grid[i][j], grid[i - di][j] + 1);
            }

            //esquerda
            for(int dj = 1; dj <= j; dj++){
                if(grid[i][j-dj] != -1) vis[grid[i][j-dj]] = 1;
                //grid[i][j] = max(grid[i][j], grid[i][j - dj] + 1);
            }

            //diagonal
            for(int d = 1; d <= min(i, j); d++){
                if(grid[i-d][j-d] != -1) vis[grid[i-d][j-d]] = 1;
                //grid[i][j] = max(grid[i][j], grid[i - d][j - d] + 1);
            }

            for(int k = 0; k < 3*MAXN; k++){
                if(!vis[k]){
                    grid[i][j] = k;
                    break;
                }
            }
        }
    }

    //print();
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    calc();

    int n;
    cin >> n;

    int ans = 0;
    for(int i = 0; i < n; i++){
        int u, v;
        cin >> u >> v;

        if(u == v || u == 0 || v == 0){
            cout << "Y\n";
            return 0;
        }

        ans ^= grid[u][v];
    }

    if(ans) cout << "Y\n";
    else cout << "N\n";

    return 0;
}
```

2.3 C

```cpp
#include<bits/stdc++.h>
using namespace std;
template<class T>
struct CoordinateCompression{
    vector<T>v;
    void push(const T &a){v.push_back(a);}
    int build(){
        sort(v.begin(),v.end());
        v.erase(unique(v.begin(),v.end()),v.end());
        return (int)v.size();
    }
    int operator[](const T &a){
        auto it=lower_bound(v.begin(),v.end(),a);
        return int(it - v.begin());
    }
};
template<class T>
struct BIT{
    int n;
    vector<T>bit;
    BIT(int n_=0):n(n_),bit(n+1){} //1-based
    void update(int x, T val){
        for(;x<=n;x+=(x&-x))bit[x]+=val;
    }
    T query(int x){ //query on [1,r]
        T ret=0;
        for(;x>0;x-=(x&-x))ret+=bit[x];
        return ret;
    }
    T query(int l, int r){ //query on [l,r]
                if(r < l)return 0;
        return query(r)-query(l-1);
    }
};
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    CoordinateCompression<int>mpx,mpy;
        int x,y;
        cin>>x>>y;
        mpx.push(0);mpy.push(0);
        mpx.push(x);mpy.push(y);

        int h,v;
        cin>>h>>v;
        vector<pair<int,int>>hori(h);
        vector<pair<int,int>>verti(v);
        for(auto &[y1,y2] : hori){
                cin>>y1>>y2;
                mpy.push(y2);
        }
        for(auto &[x1,x2] : verti){
                cin>>x1>>x2;
                mpx.push(x2);
        }
        sort(begin(hori),end(hori));
        sort(begin(verti),end(verti));
        long long ans = 1;
        {
                int len = mpy.build() + 1;
                BIT<int>edson(len + 1);
                for(int i=0;i<h;++i){
                        int y2 = hori[i].second;
                        y2 = mpy[y2];
                        ans += edson.query(y2 + 1, len) + 1;
                        edson.update(y2, 1);
                }
        }
        {
                int len = mpx.build() + 1;
                BIT<int>leo(len + 1);
                for(int i=0;i<v;++i){
                        int x2 = verti[i].second;
                        x2 = mpx[x2];
                        ans += leo.query(x2 + 1, len) + h + 1;
                        leo.update(x2, 1);
                }
        }
        cout<<ans<<'\n';
    return 0;
}
```

2.4 D

```cpp
#include "bits/stdc++.h"
using namespace std;

typedef long long ll;

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int t;
    cin >> t;

    int ans = 0;

    while(t--){
        int n;
        cin >> n;
        if(n != 1) ans++;
    }

    cout << ans << '\n';

    return 0;
}
```

2.5 E

```cpp
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    string s, t;
    cin >> s >> t;

    int n = s.size();
    int m = t.size();

    int ans = 0;
    for (int i = m - 1; i < n; i++) {
        bool possible = true;
        int start = i - m + 1;

        for (int j = start; j <= i && possible; j++)
            if (s[j] == t[j - start]) possible = false;

        if (possible) ans++;
```

```
    }

    cout << ans << "\n";
}
```

2.6 F

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

const int INF = 1e8;

int N, M;
vector<tuple<int, int, int, int>> shows;
vector<vector<int>> memo;

int songs(int i = 0, int seen = 0) {
    if (i >= M) return seen == (1 << N) - 1 ? 0 : -INF;

    int &ans = memo[i][seen];
    if (~ans) return ans;

    auto [start, end, bonus, stage] = shows[i];
    int nxt = upper_bound(shows.begin(), shows.end(), make_tuple(end, INF,
        INF, INF)) - shows.begin();

    return ans = max(songs(i + 1, seen), songs(nxt, seen | (1 << stage)) +
        bonus);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> N;
    M = 0;

    for (int i = 0; i < N; i++) {
        int Mi; cin >> Mi;
        M += Mi;

        while (Mi--) {
            int start, end, bonus;
            cin >> start >> end >> bonus;
            shows.emplace_back(start, end - 1, bonus, i);
        }
    }

    sort(shows.begin(), shows.end());

    memo.assign(M + 5, vector<int>((1 << N) + 5, -1));

    int ans = songs();
    if (ans <= 0) cout << "-1\n";
    else cout << ans << "\n";
}
```

2.7 G

```
#include "bits/stdc++.h"
using namespace std;

typedef long long ll;
typedef pair<int, int> ii;
typedef pair<ii, ll> iii;

const ll flow_inf = 1e18;
int n_stations, n_refinaria, n_edges;

vector<ll> stations;
vector<ll> refinaria;

struct FlowEdge {
    int v, u;
    int cost;
    ll cap, flow = 0;
    FlowEdge(int v, int u, ll cap, int cost) : v(v), u(u), cap(cap), cost(
        cost) {}
};

struct Dinic {
    int s, t;
    int n, m = 0;
    int limite_peso;

    queue<int> q;
    vector<int> level, ptr;
    vector<FlowEdge> edges;
    vector<vector<int> > adj;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void print(){
        for(int i = 0; i < edges.size(); i++){
            cout << edges[i].u << ' ' << edges[i].v << ' ' << edges[i].cap
                << ' ' << edges[i].flow << ' ' << edges[i].cost << '\n';
        }
    }

    void add_edge(int v, int u, ll cap, int cost) {
        edges.push_back(FlowEdge(v, u, cap, cost));
        edges.push_back(FlowEdge(u, v, 0, 0));
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while(!q.empty()) {
            int v = q.front();
            q.pop();
            for(int id : adj[v]) {
                //cout << v << ' ' << id << ' ' << edges[id].cost << '\n';
                // if(edges[id].cost > limite_peso)
                //     continue;
                if(edges[id].cap - edges[id].flow < 1)
                    continue;
                if(level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    ll dfs(int v, ll pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
```

```cpp
            int id = adj[v][cid];
            int u = edges[id].u;

            if (level[v] + 1 != level[u] || edges[id].cap - edges[id].flow < 1)
                continue;
            ll tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    ll flow() {
        ll f = 0;
        while(true){
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if(!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while(ll pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }

    void add(int max_w, int src, int snk, vector<iii> &edges_main){
        m = 0;
        adj.clear();
        edges.clear();

        adj.resize(n);

        for(int i = 1; i <= n_refinaria; i++){
            add_edge(src, i, refinaria[i], 0);
            add_edge(i, n_refinaria + i, refinaria[i], 0);
        }

        for(int i = 1; i <= n_stations; i++){
            add_edge(2*n_refinaria + i, 2*n_refinaria + n_stations + i, stations[i], 0);
            add_edge(2*n_refinaria + n_stations + i, snk, stations[i], 0);
        }

        for(int i = 0; i < n_edges; i++){
            int u = edges_main[i].first.first;
            int v = edges_main[i].first.second;
            int cost = edges_main[i].second;

            if(edges_main[i].second <= max_w)
                add_edge(u + n_refinaria, 2*n_refinaria + v, flow_inf, cost);
        }
    }
};

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n_stations >> n_refinaria >> n_edges;

    ll sum = 0;
    vector<iii> edges_main;

    refinaria.resize(n_refinaria + 1);
    stations.resize(n_stations + 1);

    for(int i = 1; i <= n_stations; i++) {
        cin >> stations[i];
        sum += stations[i];
    }

    for(int i = 1; i <= n_refinaria; i++) cin >> refinaria[i];

    int src = 0, snk = 2*n_stations + 2*n_refinaria + 1;

    Dinic gabmei(snk+1, src, snk);

    for(int i = 1; i <= n_refinaria; i++){
        gabmei.add_edge(src, i, refinaria[i], 0);
        gabmei.add_edge(i, n_refinaria + i, refinaria[i], 0);
    }

    for(int i = 1; i <= n_stations; i++){
        gabmei.add_edge(2*n_refinaria + i, 2*n_refinaria + n_stations + i, stations[i], 0);
        gabmei.add_edge(2*n_refinaria + n_stations + i, snk, stations[i], 0);
    }

    int max_weight = 0;

    for(int i = 0; i < n_edges; i++){
        int u, v, cost;
        cin >> u >> v >> cost;
        swap(u, v);
        edges_main.push_back(make_pair(make_pair(u, v), cost));
        gabmei.add_edge(u + n_refinaria, 2*n_refinaria + v, flow_inf, cost);

        max_weight = max(max_weight, cost);
    }

    gabmei.limite_peso = max_weight;

    int max_flow = gabmei.flow();

    if(max_flow < sum){
        cout << "-1\n";
        return 0;
    }

    int l = 0, r = max_weight;

    while(l < r){
        int mid = (l + r) >> 1;
        gabmei.limite_peso = mid;

        gabmei.add(mid, src, snk, edges_main);

        int flow_atual = gabmei.flow();

        if(flow_atual == max_flow) r = mid;
        else l = mid + 1;
    }

    cout << l << '\n';

    return 0;
}
```

2.8 I

```cpp
#include <bits/stdc++.h>

using namespace std;
```

```cpp
#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    cin >> N >> M;

    int L; cin >> L;
    vector<bool> lit(M);

    while (L--) {
        int X; cin >> X;
        lit[X - 1] = true;
    }

    vector<vector<int>> switches(N);
    for (int i = 0; i < N; i++) {
        int K; cin >> K;
        while (K--) {
            int Y; cin >> Y;
            switches[i].push_back(Y - 1);
        }
    }

    for (int i = 0; i <= 2 * N; i++) {
        bool all_zero = true;
        for (int j = 0; j < M && all_zero; j++)
            if (lit[j]) all_zero = false;

        if (all_zero) {
            cout << i << "\n";
            return 0;
        }

        int idx = i % N;
        for (auto l: switches[idx])
            lit[l] = !lit[l];
    }

    cout << "-1\n";
}
```

2.9 J

```cpp
#include "bits/stdc++.h"
using namespace std;

typedef long double ld;
typedef pair<ld, int> ldi;

const int MAXN = 105;
const int MAXK = 1030;
const ld INF = 1e18 + 5;

int n, k;
ld dist[MAXK][MAXN*MAXN];
vector<vector<pair<int, ld> > > adj;

void floyd(){
    for(int k = 0; k < n; k++){
        for(int i = 0; i < n; i++){
            for(int j = 0; j < n; j++){
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}

ld euclidean_dist(pair<int, int> &a, pair<int, int> &b){
    ld x = a.first - b.first;
    ld y = a.second - b.second;

    return sqrt(x*x + y*y);
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    cout << setprecision(5) << fixed;

    cin >> n >> k;

    vector<pair<int, int> > edges(n);

    for(int i = 0; i < n; i++)
        cin >> edges[i].first >> edges[i].second;

    for(int i = 0; i < MAXN; i++)
        for(int j = 0; j < MAXN; j++)
            dist[i][j] = INF;

    for(int i = 0; i < n; i++){
        dist[i][i] = 0;
        for(int j = k; j < n; j++){
            dist[i][j] = euclidean_dist(edges[i], edges[j]);
        }
    }

    floyd();

    //Compute Steiner Trees
    vector<vector<ld> > minDist(1 << k, vector<ld> (n, INF));

    for(int i = 0; i < k; i++) minDist[1 << i][i] = 0;

    for(int mask = 0; mask < (1 << k); mask++){
        for(int i = 0; i < n; i++){
            for(int ss = mask; ss > 0; ss = (ss - 1) & mask){    //compute all possible subtrees being joined in rooted i
                minDist[mask][i] = min(minDist[mask][i], minDist[ss][i] + minDist[mask - ss][i]);
            }
            if(minDist[mask][i] < INF){
                for(int j = 0; j < n; j++){
                    minDist[mask][j] = min(minDist[mask][j], minDist[mask][i] + dist[i][j]);
                }
            }
        }
    }

    vector<ld> minTotal(1 << k, INF);
    for(int mask = 0; mask < (1<<k); mask++)
        for(int i = 0; i < n; i++)
            minTotal[mask] = min(minTotal[mask], minDist[mask][i]);

    cout << minTotal[(1<<k) - 1] << '\n';

    return 0;
}

//rooted in i and mask terminals nodes have been joined
```

2.10 L

```cpp
#include<bits/stdc++.h>
using namespace std;
```

```cpp

struct node{
    int x;
    node(int _x=0):x(_x){}
        node operator+(const node& o){ // merge function
                return node(x+o.x);
        }
        bool operator!=(const node& o){
                return o.x != x;
        }
}nil(0),nada(-1);

struct SegmentTree{
        int n;
        vector<node>tree,lazy;
        SegmentTree(int _n=0):n(_n),tree(4*n,nil),lazy(4*n,nada){}

        void calc(node& a, const node& b, int len){
                a= node(b.x*len);
        }

        void push(int pos, int tl,int tr){ //assignment update, sum query
                if(lazy[pos]!=nada){
                        int esq=2*pos, dir=2*pos+1;
                        int mid = (tl+tr)/2;
                        calc(tree[esq],lazy[pos],mid-tl+1);
                        calc(tree[dir],lazy[pos],tr-(mid+1)+1);

                        lazy[esq]=lazy[dir]=lazy[pos];
                        lazy[pos]=nada;
                }
        }
        void update(int pos,int tl,int tr,int l, int r, node x){
                int esq=2*pos, dir=2*pos+1;
                int mid = (tl+tr)/2;
                if(tl>r || tr<l){
                        return;
                }else if(tl>=l && tr<=r){
                        calc(tree[pos],x,tr-tl+1);
                        lazy[pos]=x;
                }else{
                        push(pos,tl,tr);
                        update(esq,tl,mid,l,r,x);
                        update(dir,mid+1,tr,l,r,x);

                        tree[pos]=tree[esq]+tree[dir];
                }
        }

        void update(int l, int r, node x){
                update(1,0,n-1,l,r, x);
        }
        node query(int pos, int tl, int tr, int l, int r){
                int esq=2*pos, dir=2*pos+1;
                int mid = (tl+tr)/2;
                if(tl>r || tr<l){
                        return nil;//depends of the query type
                }else if(tl>=l && tr<=r){
                        return tree[pos];
                }else{
                        push(pos,tl,tr);
                        return query(esq,tl,mid,l,r) + query(dir,mid+1,tr,l,r);
                }
        }
        node query(int l, int r){
                return query(1,0,n-1,l,r);
        }
};

vector<vector<int>> adj;
template<class T, class F>
struct HeavyLight{
        vector<int>pos,pai,h,sz;
        T tree;
        int timer;

        HeavyLight(int n, int root=0):pos(n),pai(n),h(n),sz(n),tree(n){
                dfs(root);
                timer = 0;
                h[root]=root;
                buildHLD(root);
        }
        void dfs(int u, int p=-1){
                sz[u] = 1;
                for(int &v:adj[u]){
                        if(v!=p){
                                dfs(v,u);
                                sz[u]+=sz[v];
                                if(sz[v] > sz[adj[u][0]] || adj[u][0] == p) swap(v,adj[u][0]);
                        }
                }
        }
        void buildHLD(int u, int p=-1){
                pos[u]=timer++;
                for(int v:adj[u]){
                        if(v!=p){
                                pai[v]=u;
                                h[v] =(v==adj[u][0] ? h[u] : v);
                                buildHLD(v,u);
                        }
                }
        }
        F queryPath(int a ,int b){
                if(pos[a]<pos[b])swap(a,b);

                if(h[a]==h[b])return tree.query(pos[b],pos[a]);

                return tree.query(pos[h[a]],pos[a]) + queryPath(pai[h[a]],b);
        }
        void updatePath(int a, int b, F x){
                if(pos[a]<pos[b])swap(a,b);

                if(h[a]==h[b]){
                        tree.update(pos[b],pos[a],x);
                        return;
                }
                tree.update(pos[h[a]],pos[a],x);
                updatePath(pai[h[a]],b,x);
        }
};

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
        int n, q;
        cin>>n>>q;
        adj.assign(n,{});
        for(int i=0;i<n-1;++i){
                int u,v;
                cin>>u>>v;
                adj[--u].push_back(--v);
                adj[v].push_back(u);
        }
    HeavyLight<SegmentTree,node>hld(n);
        while(q--){
                int a,b,c,d;
                cin>>a>>b>>c>>d;
                --a;--b;--c;--d;
                hld.updatePath(a,b,node(1));
                int cnt2 = hld.queryPath(c,d).x;
                cout<<cnt2<<'\n';
                hld.updatePath(a,b,node(0));
```

```
        }
    return 0;
}
```

3 2019

3.1 A

```
#include <bits/stdc++.h>

using namespace std;

struct DSU {
private:
    int n;
    vector<int> ds, sz;

public:
    DSU(int n) {
        this->n = n;
        ds.resize(n);
        sz.resize(n);

        for (int i = 0; i < n; i++)
            ds[i] = i, sz[i] = 1;
    }

    int find(int i) {
        if (ds[i] == i) return i;
        return ds[i] = find(ds[i]);
    }

    void join(int i, int j) {
        i = find(i); j = find(j);
        if (sz[i] < sz[j]) swap(i, j);
        if (i != j) sz[i] += sz[j];
        ds[j] = i;
    }
};

long double dist(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1;
    int dy = y2 - y1;
    return sqrt(dx * dx + dy * dy);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int M, N, K;
    cin >> M >> N >> K;

    vector<tuple<int, int, int>> sensors(K);
    for (int i = 0; i < K; i++) {
        int X, Y, S;
        cin >> X >> Y >> S;
        sensors[i] = make_tuple(X, Y, S);
    }

    DSU tree(K);
    for (int i = 0; i < K; i++)
        for (int j = 0; j < K; j++)
            if (i != j) {
                auto [xi, yi, si] = sensors[i];
                auto [xj, yj, sj] = sensors[j];

                if (dist(xi, yi, xj, yj) <= si + sj)
                    tree.join(i, j);
            }

    map<int, vector<int>> components;
    for (int i = 0; i < K; i++) {
        int root = tree.find(i);
        components[root].push_back(i);
    }

    for (auto [root, component]: components) {
        bool up = false, right = false, down = false, left = false;
        for (auto i: component) {
            auto [x, y, s] = sensors[i];
            up |= x - s <= 0;
            right |= y + s >= N;
            down |= x + s >= M;
            left |= y - s <= 0;
        }

        bool ans = up && down;
        ans |= up && left;
        ans |= right && down;
        ans |= right && left;

        if (ans) {
            cout << "N\n";
            return 0;
        }
    }

    cout << "S\n";
}
```

3.2 B

```
#include "bits/stdc++.h"
using namespace std;

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int N;
    cin >> N;

    int max_ = 0;
    cin >> max_;

    for(int i = 1; i < N; i++){
        int a;
        cin >> a;
        if(a > max_){
            cout << "N\n";
            return 0;
        }
    }

    cout << "S\n";

    return 0;
}
```

3.3 D

```
#include <bits/stdc++.h>
using namespace std;

```

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef long long int ll;
typedef double ld;
typedef pair<int, int> pii;
typedef vector<int> vi;

#define endl '\n'
#define PB push_back
#define F first
#define S second
#define ALL(x) x.begin(), x.end()

typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> ordered_set;

vector<vi> adj, revAdj;
vi vis;

stack<int> bfs(int source) {
  queue<int> q;
  q.push(source);

  stack<int> leafs;
  while(!q.empty()) {
    int cur = q.front(); q.pop();
    if(vis[cur]) continue;
    vis[cur] = true;

    if(revAdj[cur].size() == 0) leafs.push(cur);

    for(auto next: revAdj[cur]) {
      q.push(next);
    }
  }
  return leafs;
}

int dfs(int cur) {
  vis[cur] = true;
  for(auto next: adj[cur]) {
    if(vis[next]) continue;
    return dfs(next) + 1;
  }

  return 1;
}

int main() {
  cin.tie(0);
  ios_base::sync_with_stdio(false);

  int n, k;
  cin >> n >> k;

  adj.resize(n);
  revAdj.resize(n);
  vis.resize(n);

  int parent;
  for(int i = 1; i < n; ++i) {
    cin >> parent;
    --parent;
    adj[i].PB(parent);
    revAdj[parent].PB(i);
  }

  auto leafs = bfs(0);
  priority_queue<int, vi> pq;

  vis.assign(n, 0);

  while(!leafs.empty()) {
    int cur = leafs.top(); leafs.pop();
    int dist = dfs(cur);
    pq.push(dist);
  }

  int ans = 0;
  while(k-- && !pq.empty()) {
    ans += pq.top(); pq.pop();
  }
  cout << ans << endl;

  return 0;
}
```

3.4 G

```cpp
#include "bits/stdc++.h"
using namespace std;

typedef long double ld;

const ld INF = 1e18;

void hungarian(vector<vector<ld> > &A, int n, int m){
    vector<ld> u(n+1), v(m+1), p(m+1), way(m+1);

    for(int i = 1; i <= n; ++i){
        p[0] = i;
        int j0 = 0;
        vector<ld> minv (m+1, INF);
        vector<bool> used (m+1, false);

        do{
            used[j0] = true;
            int i0 = p[j0], j1;
            ld delta = INF;
            for(int j = 1; j <= m; ++j){
                if(!used[j]) {
                    ld cur = A[i0][j] - u[i0] - v[j];
                    if(cur < minv[j])
                        minv[j] = cur,  way[j] = j0;
                    if(minv[j] < delta)
                        delta = minv[j],  j1 = j;
                }
            }

            for(int j = 0; j <= m; ++j){
                if(used[j]){
                    u[p[j]] += delta,  v[j] -= delta;
                }
                else{
                    minv[j] -= delta;
                }
            }

            j0 = j1;
        } while(p[j0] != 0);

        do{
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }

    for(int i = 1; i <= m; ++i) cout << p[i] << ' ';

}
```

```cpp

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;

    vector<vector<ld> > A(n+1, vector<ld>(n+1));

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            cin >> A[i][j];
            A[i][j] = -log(A[i][j]);
        }
    }

    hungarian(A, n, n);

    return 0;
}
```

3.5 H

```cpp
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int V, N;
    cin >> V >> N;

    int total = V * N;
    for (int i = 1; i < 10; i++) {
        if (i > 1) cout << " ";

        int ans = (total * i + 9) / 10;
        cout << ans;
    }

    cout << "\n";
}
```

3.6 I

```cpp
#include "bits/stdc++.h"
using namespace std;

typedef long long ll;
typedef pair<int, int> ii;

const int MAXN = 405;
const int MAXQ = 1e5+5;
const ll INF = 1e18 + 5;

int n, r;
ll tmp[MAXN][MAXN];
ll dist[MAXN][MAXN];

void read(){
    for(int i = 0; i < MAXN; i++){
        for(int j = 0; j < MAXN; j++){
            if(i == j) dist[i][j] = 0;
            else dist[i][j] = INF;
        }
    }

    for(int i = 0; i < r; i++){
        ll t;
        int u, v;
        cin >> u >> v >> t;
        dist[u][v] = dist[v][u] =  min(dist[u][v], t);
    }
}

void copy(){
    for(int i = 0; i < MAXN; i++){
        for(int j = 0; j < MAXN; j++){
            tmp[i][j] = dist[i][j];
        }
    }
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> r;

    vector<pair<ll, int> > planets(n+5);
    for(int i = 1; i <= n; i++){
        cin >> planets[i].first;
        planets[i].second = i;
    }

    read();

    sort(planets.begin() + 1, planets.begin() + 1 + n);

    vector<pair<int, ii> > query_low[MAXN];
    vector<pair<int, ii> > query_high[MAXN];

    int q;
    cin >> q;
    for(int i = 1; i <= q; i++){
        int u, v, k, t, cnt = 0;
        cin >> u >> v >> k >> t;

        if(t){  //Mais quentes
            for(int j = n; j > 0; j--){
                while(j > 1 && planets[j].first == planets[j-1].first) j--;
                if(++cnt >= k){
                    query_high[j].push_back(make_pair(i, make_pair(u, v)));
                    break;
                }
            }
            if(cnt < k) query_high[1].push_back(make_pair(i, make_pair(u, v
                )));
        }
        else{   //Mais frios
            for(int j = 1; j <= n; j++){
                while(j < n && planets[j].first == planets[j+1].first) j++;
                if(++cnt >= k){
                    query_low[j].push_back(make_pair(i, make_pair(u, v)));
                    break;
                }
            }
            if(cnt < k) query_low[n].push_back(make_pair(i, make_pair(u, v)
                ));
        }
    }

    vector<ll> ans(MAXQ);

    copy();

```

```cpp
    for(int k = 1; k <= n; k++){
        int id = planets[k].second;
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++){
                tmp[i][j] = tmp[j][i] = min(tmp[i][j], tmp[i][id] + tmp[id][j]);
            }
        }
        for(auto x: query_low[k]){
            ans[x.first] = tmp[x.second.first][x.second.second];
        }
    }

    copy();

    for(int k = n; k >= 1; k--){
        int id = planets[k].second;
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++){
                tmp[i][j] = tmp[j][i] = min(tmp[i][j], tmp[i][id] + tmp[id][j]);
            }
        }
        for(auto x: query_high[k]){
            ans[x.first] = tmp[x.second.first][x.second.second];
        }
    }

    for(int i = 1; i <= q; i++){
        if(ans[i] < INF) cout << ans[i] << '\n';
        else cout << "-1\n";
    }
}
```

3.7 J

```cpp
#include "bits/stdc++.h"
using namespace std;

struct node{
    int cartas[13];
    bool temCoringa, podePassar;
};

vector<node> jogador;

bool venceu(int i){
    for(int j = 0; j < 13; j++){
        if(jogador[i].cartas[j] == 4) return true;
    }
    return false;
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, k;
    cin >> n >> k;

    jogador.resize(n);

    for(int i = 0; i < n; i++){
        string s;
        cin >> s;
        for(int t = 0; t < 13; t++){
            jogador[i].cartas[t] = 0;
        }
        jogador[i].temCoringa = false;
        jogador[i].podePassar = true;
        for(int j = 0; j < s.length(); j++){
            char c = s[j];
            if(c == 'A') jogador[i].cartas[0]++;
            else if(c == 'D') jogador[i].cartas[9]++;
            else if(c == 'Q') jogador[i].cartas[10]++;
            else if(c == 'J') jogador[i].cartas[11]++;
            else if(c == 'K') jogador[i].cartas[12]++;
            else jogador[i].cartas[c - '1']++;
        }
    }

    for(int i = 0; i < n; i++){
        if(venceu(i) && i+1 != k){
            cout << i+1 << '\n';
            return 0;
        }
    }

    int atual = k-1, nxt;

    jogador[atual].temCoringa = true;
    jogador[atual].podePassar = false;

    while(1){
        nxt = (atual + 1) % n;
        if(jogador[atual].temCoringa && jogador[atual].podePassar){
            jogador[atual].temCoringa = false;
            jogador[atual].podePassar = true;

            jogador[nxt].temCoringa = true;
            jogador[nxt].podePassar = false;
        }
        else{
            jogador[atual].podePassar = true;
            int min_ = 50, idx = -1;
            for(int i = 0; i < 13; i++){
                if(jogador[atual].cartas[i] < min_ && jogador[atual].cartas[i] != 0){
                    idx = i;
                    min_ = jogador[atual].cartas[i];
                }
            }
            jogador[atual].cartas[idx]--;
            jogador[nxt].cartas[idx]++;
        }
        if(venceu(atual)){
            cout << atual + 1 << '\n';
            return 0;
        }
        atual = nxt;
    }

    return 0;
}
```

3.8 K

```cpp
#include "bits/stdc++.h"
using namespace std;

#define int long long

const int m = 4;
const int MOD = 1e9 + 7;

class Matrix{
    public:
        int mat[m][m];
        Matrix operator * (const Matrix &p){
            Matrix ans;
```

```
            for(int i = 0; i < m; i++)
                for(int j = 0; j < m; j++){
                    ans.mat[i][j] = 0;
                    for(int k = ans.mat[i][j] = 0; k < m; k++)
                        ans.mat[i][j] = (ans.mat[i][j] + (mat[i][k] % MOD)
                            * (p.mat[k][j] % MOD)) % MOD;
                }
            return ans;
        }
};

Matrix aux = {2, 4, 0, 2, 1, 0, 0, 0, 0, 8, 2, 0, 0, 0, 0, 2};

Matrix fexp(Matrix ans, int b){
    while(b){
        if(b & 1) ans = aux*ans;
        aux = aux*aux;
        b >>= 1;
    }

    return ans;
}

int32_t main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;

    if(n == 1) cout << "2\n";
    else{
        Matrix ans;
        ans.mat[0][0] = 6;
        ans.mat[1][0] = 1;
        ans.mat[2][0] = 0;
        ans.mat[3][0] = 2;

        ans = fexp(ans, n - 2);
        int sum = (4*ans.mat[0][0] % MOD + 2*ans.mat[2][0] % MOD) % MOD;
        cout << sum << '\n';
    }
}
```

3.9 L

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

long count(long N, long pot) {
    if (pot == 1LL) return 2LL;

    pot >>= 1LL;
    if (N <= pot) return count(N, pot);
    else {
        long ans = count(N - pot, pot);
        ans <<= 1LL;
        return ans;
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    long N; cin >> N;

    long pot = 1LL, sum = 1LL;
    while (sum < N) {
        pot <<= 1LL;
        sum += pot;
    }

    sum -= pot;
    long ans = count(N - sum, pot);
    cout << ans << "\n";
}
```

3.10 M

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, C, T;
    cin >> N >> C >> T;

    vector<int> P(N);
    int sum = 0, mx = INT_MIN;

    for (int i = 0; i < N; i++) {
        cin >> P[i];
        sum += P[i];
        mx = max(mx, P[i]);
    }

    int l = mx, r = sum, ans;
    while (l <= r) {
        int mid = (l + r) / 2;

        int cnt = 1, curr_sum = 0;
        for (int i = 0; i < N; i++) {
            if (curr_sum + P[i] <= mid) curr_sum += P[i];
            else {
                cnt++;
                curr_sum = P[i];
            }
        }

        if (cnt > C) l = mid + 1;
        else r = mid - 1, ans = mid;
    }

    ans = (ans + T - 1) / T;
    cout << ans << "\n";
}
```

4 2020

4.1 A

```
#include <bits/stdc++.h>

using namespace std;

#define ldouble long double

int main() {
    ios_base::sync_with_stdio(false);
```

```
    cin.tie(NULL);

    cout << setprecision(6) << fixed;

    int N, A, B;
    cin >> N >> A >> B;

    bool zero = false;
    if (!A) {
        A++;
        zero = true;
    }

    vector<ldouble> dp(N + 1, 0.0L), ps(N + 1, 0.0L);
    for (int i = 1; i <= N; i++) {
        ldouble sum = (i - A >= 0 ? ps[i - A] : ps[0]);
        sum -= (i - B > 0 ? ps[i - B - 1] : ps[0]);

        dp[i] = 1.0L / (B - A + 1.0L) * sum + 1.0L;
        if (zero) dp[i] += 1.0L / B;
        ps[i] = ps[i - 1] + dp[i];
    }

    ldouble ans = dp[N];
    cout << ans << "\n";
}
```

4.2 B

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

const int MAX = 10;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;
    bool vis[MAX][MAX] = {0};

    int sum = 0;
    while (N--) {
        int D, L, R, C;
        cin >> D >> L >> R >> C;

        if (!D) {
            for (int i = C - 1; i < C + L - 1 && i < MAX; i++)
                vis[R - 1][i] = true;
        } else {
            for (int i = R - 1; i < R + L - 1 && i < MAX; i++)
                vis[i][C - 1] = true;
        }

        sum += L;
    }

    int cnt = 0;
    for (int i = 0; i < MAX; i++)
        for (int j = 0; j < MAX; j++)
            cnt += vis[i][j];

    cout << (sum == cnt ? 'Y' : 'N') << "\n";
}
```

4.3 E

```
#include <bits/stdc++.h>

using namespace std;

const int LOG = 20;
const int MAX = 1e5 + 5;

struct FenwickTree {
private:
    int n;
    vector<int> tree;

public:
    FenwickTree() {}
    FenwickTree(int n) {
        this->n = n + 1;
        tree.assign(n + 1, 0);
    }

    void add(int i, int v) {
        if (!i) tree[0] += v;
        else {
            while (i < n) {
                tree[i] += v;
                i += LSO(i);
            }
        }
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

private:
    int LSO(int i) {
        return i & -i;
    }

    int sum(int i) {
        if (i < 0) return 0;

        int ans = tree[0];
        while (i) {
            ans += tree[i];
            i -= LSO(i);
        }

        return ans;
    }
};

vector<int> A;
vector<vector<int>> tree;
vector<vector<int>> up, Ls;
FenwickTree solver;
vector<int> ans;

int kth_ancestor(int node, int k) {
    for (int i = 0; i < LOG; i++)
        if (k & (1 << i))
            node = up[i][node];

    return node;
}

void solve(int node) {
    for (int L: Ls[node])
        solver.add(L, 1);
```

```
    ans[node] = solver.sum(1, A[node]);
    for (int child: tree[node])
        solve(child);

    for (int L: Ls[node])
        solver.add(L, -1);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    cin >> N >> M;

    A.resize(N);
    vector<int> B(N);
    tree.assign(N, vector<int>());

    int root = -1;
    for (int i = 0; i < N; i++) {
        cin >> A[i] >> B[i];
        if (i == --B[i]) root = i;
        else tree[B[i]].push_back(i);
    }

    assert(root != -1);

    up.assign(LOG, vector<int>(N));
    for (int i = 0; i < N; i++)
        up[0][i] = B[i];

    for (int i = 1; i < LOG; i++)
        for (int j = 0; j < N; j++)
            up[i][j] = up[i - 1][up[i - 1][j]];

    Ls.assign(N, vector<int>());
    for (int i = 0; i < M; i++) {
        int O, L, R;
        cin >> O >> L >> R;

        int l = 1, r = N, lift = 0;
        while (l <= r) {
            int mid = (l + r) / 2;

            int ancestor = kth_ancestor(O - 1, mid);
            if (A[ancestor] > R) r = mid - 1;
            else l = mid + 1, lift = mid;
        }

        Ls[kth_ancestor(O - 1, lift)].push_back(L);
    }

    solver = FenwickTree(MAX);
    ans.resize(N);
    solve(root);

    for (int i = 0; i < N; i++) {
        if (i) cout << " ";
        cout << ans[i];
    }

    cout << "\n";
}
```

4.4 F

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    string s; cin >> s;

    bool won = false;
    int g[2] = {0}, p[2] = {0}, server = 0;

    for (char c: s) {
        if (c == 'Q') {
            if (!won) cout << g[0] << " (" << p[0] << (server == 0 ? "*" : "") << ") - " << g[1] << " (" << p[1] << (server == 1 ? "*" : "") << ")\n";
            else {
                cout << g[0] << " ";
                if (g[0] > g[1]) cout << "(winner) ";
                cout << "- " << g[1];
                if (g[1] > g[0]) cout << " (winner)";
                cout << "\n";
            }
        } else {
            if (won) continue;

            int point = c == 'S' ? server : 1 - server;
            p[point]++;

            if (p[point] == 10 || (p[point] >= 5 && p[point] >= p[1 - point] + 2)) g[point]++, p[0] = p[1] = 0;
            if (g[point] == 2) won = true;
            server = point;
        }
    }
}
```

4.5 G

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int C; cin >> C;

    int ans = 100, curr = 100;
    for (int i = 0; i < C; i++) {
        int V; cin >> V;
        curr += V;
        ans = max(ans, curr);
    }

    cout << ans << "\n";
}
```

4.6 H

```
#include <bits/stdc++.h>

using namespace std;

#define long long long int
```

```cpp

const int MAX = 51;

int N, K;
vector<long> W, ps;
long comb[MAX][MAX];
long A, B;

long solve(int i = 0, int k = 0, long sum = 0LL) {
    if (sum > B) return 0LL;
    if (k >= K) return sum >= A;
    if (i >= N || sum + ps[N] - ps[i] < A) return 0LL;
    if (sum >= A && sum + ps[N] - ps[i] <= B) return comb[N - i][K - k];
    return solve(i + 1, k, sum) + solve(i + 1, k + 1, sum + W[i]);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> N >> K;

    W.resize(N);
    for (int i = 0; i < N; i++)
        cin >> W[i];

    sort(W.rbegin(), W.rend());

    ps.assign(N + 1, 0);
    for (int i = 1; i <= N; i++)
        ps[i] = ps[i - 1] + W[i - 1];

    cin >> A >> B;

    for (int i = 0; i < MAX; i++)
        comb[i][0] = comb[i][i] = 1LL;

    for (int i = 1; i < MAX; i++)
        for (int j = 1; j < i; j++)
            comb[i][j] = comb[i - 1][j - 1] + comb[i - 1][j];

    long ans = solve();
    cout << ans << "\n";
}
```

4.7 I

```cpp
#include <bits/stdc++.h>

using namespace std;

const int MOD = 1e9 + 7;

vector<vector<int>> tree;
vector<vector<int>> memo;

int solutions(bool know = false, int node = 0) {
    if (tree[node].empty()) return 1;

    int &ans = memo[know][node];
    if (~ans) return ans;

    int sz = tree[node].size();
    vector<int> pprod(sz + 1, 1);

    for (int i = 1; i <= sz; i++)
        pprod[i] = (1LL * pprod[i - 1] * solutions(false, tree[node][i - 1])) % MOD;

    vector<int> sprod(sz + 2, 1);
    for (int i = sz; i > 0; i--)
        sprod[i] = (1LL * sprod[i + 1] * solutions(false, tree[node][i - 1])) % MOD;

    ans = 0;
    for (int i = 1; i <= sz; i++) {
        int nprod = (1LL * pprod[i - 1] * sprod[i + 1]) % MOD;
        nprod = (1LL * nprod * solutions(true, tree[node][i - 1])) % MOD;
        ans = (ans + nprod) % MOD;
    }

    if (!know) ans = (ans + pprod[sz]) % MOD;

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    tree.assign(N, vector<int>());
    for (int i = 1; i < N; i++) {
        int P; cin >> P;
        tree[P - 1].push_back(i);
    }

    memo.assign(2, vector<int>(N + 5, -1));

    int ans = solutions();
    cout << ans << "\n";
}
```

4.8 L

```cpp
#include <bits/stdc++.h>

using namespace std;

int L, C;
vector<string> grid;
vector<vector<vector<bool>>> part;

string build(int i, int j, int n, int row, int col) {
    string t = "";
    for (int k = 0; k < n; k++)
        t += grid[i + row * k][j + col * k];

    sort(t.begin(), t.end());
    return t;
}

void mark(int i, int j, int n, int row, int col, int idx) {
    for (int k = 0; k < n; k++)
        part[i + row * k][j + col * k][idx] = true;
}

void find(string s, int idx) {
    int n = s.size();
    sort(s.begin(), s.end());

    for (int i = 0; i < L; i++)
        for (int j = 0; j < C; j++) {
            if (C - j >= n) {
                string t = build(i, j, n, 0, 1);
                if (s == t) mark(i, j, n, 0, 1, idx);
            }

            if (L - i >= n) {
```

```
                string t = build(i, j, n, 1, 0);
                if (s == t) mark(i, j, n, 1, 0, idx);
            }

            if (C - j >= n && L - i >= n) {
                string t = build(i, j, n, 1, 1);
                if (s == t) mark(i, j, n, 1, 1, idx);
            }

            if (j + 1 >= n && L - i >= n) {
                string t = build(i, j, n, 1, -1);
                if (s == t) mark(i, j, n, 1, -1, idx);
            }
        }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> L >> C;

    grid.resize(L);
    for (int i = 0; i < L; i++)
        cin >> grid[i];

    int N; cin >> N;

    part.assign(L, vector<vector<bool>>(C, vector<bool>(N, false)));
    for (int i = 0; i < N; i++) {
        string word;
        cin >> word;
        find(word, i);
    }

    int ans = 0;
    for (int i = 0; i < L; i++)
        for (int j = 0; j < C; j++) {
            int cnt = 0;
            for (int k = 0; k < N; k++)
                cnt += part[i][j][k];

            if (cnt > 1) ans++;
        }

    cout << ans << "\n";
}
```

4.9 N

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 4e7 + 5;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int M, N, K;
    cin >> M >> N >> K;

    vector<llong> c(N);
    for (int i = 0; i < N; i++)
        cin >> c[i];

    vector<int> deg(N, 0);
    vector<vector<int>> g(M, vector<int>());

    for (int i = 0; i < K; i++) {
        int m, n, d;
        cin >> m >> n >> d;

        deg[n - 1] += d;
        g[m - 1].push_back(n - 1);
    }

    llong prime = 2LL;
    vector<llong> ans(M);

    for (int i = 0; i < M; i++) {
        int n = g[i][0];
        if (deg[n] == 1) prime = c[n];
        else {
            while (c[n] % prime) prime++;
            assert(prime < MAX);
        }

        for (int n: g[i])
            while (!(c[n] % prime)) {
                deg[n]--;
                c[n] /= prime;
            }

        ans[i] = prime;
    }

    for (int i = 0; i < M; i++) {
        if (i) cout << " ";
        cout << ans[i];
    }

    cout << "\n";
}
```

5 2021

5.1 A

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 5e3 + 5;
const int MOD = 1e9 + 7;

int invs[MAX];

int exp(int a, int n) {
    if (!n) return 1;

    int ans = exp(a, n >> 1);
    ans = (1LL * ans * ans) % MOD;

    if (!(n & 1)) return ans;
    return (1LL * ans * a) % MOD;
}

int mod_mult_inv(int a, int m) {
    return exp(a, m - 2);
}

int interpolate(int x, vector<int> &ys) {
    int n = (int) ys.size() - 1;
```

```
    vector<int> pnum(n + 2);
    vector<int> snum(n + 2);

    pnum[0] = 1;
    for (int i = 1; i <= n; i++)
        pnum[i] = (((1LL * pnum[i - 1] * (x - i)) % MOD) + MOD) % MOD;

    snum[n + 1] = 1;
    for (int i = n; i > 0; i--)
        snum[i] = (((1LL * snum[i + 1] * (x - i)) % MOD) + MOD) % MOD;

    int ans = 0;
    for (int i = 1; i <= n; i++) {
        int num = (1LL * pnum[i - 1] * snum[i + 1]) % MOD;
        int den = (1LL * invs[n - i] * invs[i - 1]) % MOD;
        if ((n - i) & 1) den = (MOD - den) % MOD;

        int add = (1LL * num * den) % MOD;
        add = (1LL * add * ys[i]) % MOD;
        ans = (ans + add) % MOD;
    }

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int fact = invs[0] = 1;
    for (int i = 1; i < MAX; i++) {
        fact = (1LL * fact * i) % MOD;
        invs[i] = mod_mult_inv(fact, MOD);
    }

    int N, R;
    cin >> N >> R;

    vector<int> p(N);
    for (int i = 0; i < N; i++)
        cin >> p[i];

    vector<int> ys(3, 0);
    ys[1] = 1;
    ys[2] = 2;

    int mn = INT_MIN;
    for (int i = N - 1; i >= 0; i--) {
        mn = max(mn, p[i]);
        int rem = interpolate(mn - 1, ys);
        ys.push_back(interpolate(N - i + 2, ys));

        for (int j = 1; j <= N - i + 2; j++) {
            ys[j] = (ys[j] + ys[j - 1]) % MOD;
            ys[j] = (ys[j] - rem + MOD) % MOD;
        }
    }

    if (R < mn) cout << "0\n";
    else {
        int ans = interpolate(R, ys);
        ans = (ans - interpolate(R - 1, ys) + MOD) % MOD;
        cout << ans << "\n";
    }
}
```

5.2 B

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

struct RabinKarp {
private:
    string s;
    int n, base, mod;
    vector<llong> p, h;

public:
    RabinKarp() {}
    RabinKarp(string &s, int base = 243, int mod = 1e9 + 7) {
        this->s = s;
        this->n = s.size();

        this->base = base;
        this->mod = mod;

        build();
    }

    int hash(int l, int r) {
        if (!l) return h[r];
        return (h[r] - ((h[l - 1] * p[r - l + 1]) % mod) + mod) % mod;
    }

    int size() {
        return n;
    }

private:
    void build() {
        p.assign(n + 1, 1);
        for (int i = 1; i <= n; i++)
            p[i] = (p[i - 1] * base) % mod;

        h.assign(n, s[0]);
        for (int i = 1; i < n; i++)
            h[i] = (h[i - 1] * base + s[i]) % mod;
    }
};

struct Hasher {
private:
    RabinKarp h1, h2;

public:
    Hasher() {}
    Hasher(string &s) {
        h1 = RabinKarp(s);
        h2 = RabinKarp(s, 243, 1e9 + 9);
    }

    int size() {
        return h1.size();
    }

    pair<int, int> hash(int l, int r) {
        return make_pair(h1.hash(l, r), h2.hash(l, r));
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    cin >> N >> M;

    string A; cin >> A;

```

```
    vector<Hasher> hashers(M);
    for (int i = 0; i < M; i++) {
        string S; cin >> S;
        hashers[i] = Hasher(S);
    }

    string S = A + A;
    Hasher ahasher(S);

    int l = 1, r = N, ans = 0;
    while (l <= r) {
        int mid = (l + r) / 2;

        vector<pair<int, int>> hashes;
        for (int i = 0; i < M; i++)
            for (int j = mid - 1; j < hashers[i].size(); j++)
                hashes.push_back(hashers[i].hash(j - mid + 1, j));

        sort(hashes.begin(), hashes.end());
        hashes.erase(unique(hashes.begin(), hashes.end()), hashes.end());

        vector<int> cnt(N);
        for (int i = 0; i < 2 * N - mid + 1; i++)
            if (binary_search(hashes.begin(), hashes.end(), ahasher.hash(i,
                  i + mid - 1))) {
                int l = max(i + mid - N, 0), r = i + 1;
                if (l < N) cnt[l]++;
                if (r < N) cnt[r]--;
            }

        bool all = cnt[0] > 0;
        for (int i = 1; i < N && all; i++) {
            cnt[i] += cnt[i - 1];
            if (!cnt[i]) all = false;
        }

        if (!all) r = mid - 1;
        else l = mid + 1, ans = mid;
    }

    cout << ans << "\n";
}
```

5.3 C

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int exp(int a, int n, int m) {
    if (!n) return 1 % m;

    int ans = exp(a, n >> 1, m);
    ans = (1LL * ans * ans) % m;

    if (!(n & 1)) return ans;
    return (1LL * ans * a) % m;
}

void extended_euclidean(llong a, llong b, llong &x, llong &y) {
    if (!b) {
        x = 1LL;
        y = 0LL;
    } else {
        extended_euclidean(b, a % b, x, y);
        llong aux = x; x = y;
        y = aux - a / b * y;
    }
}

llong mod_mult_inv(llong a, llong m) {
    llong x, y;
    extended_euclidean(a, m, x, y);
    return (x % m + m) % m;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int B, L;
    cin >> B >> L;

    int sum = 0;
    vector<int> D(L);

    for (int i = 0; i < L; i++) {
        cin >> D[i];

        int add = (1LL * D[i] * exp(B, L - i - 1, B + 1)) % (B + 1);
        sum = (sum + add) % (B + 1);
    }

    if (!sum) {
        cout << "0 0\n";
        return 0;
    }

    bool done = false;
    int idx = -1, digit = -1;

    for (int i = 0; i < L && !done; i++) {
        int power = exp(B, L - i - 1, B + 1);
        int inv = mod_mult_inv(power, B + 1);
        int diff = (1LL * sum * inv) % (B + 1);

        int nd = D[i] - diff;
        if (nd >= 0 && nd < D[i]) {
            digit = nd;
            idx = i + 1;
            done = true;
        }
    }

    cout << idx << " " << digit << "\n";
}
```

5.4 D (wrong)

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

struct DSU {
private:
    int n;
    bool bipartite;
    vector<int> ds, sz, c;
    vector<vector<int>> children;

public:
    DSU(int n) {
        this->n = n;
        bipartite = true;

        c.assign(n, 0);
```

```
        sz.assign(n, 1);

        ds.resize(n);
        iota(ds.begin(), ds.end(), 0);

        children.assign(n, vector<int>(1));
        for (int i = 0; i < n; i++)
            children[i][0] = i;
    }

    bool is_bipartite() {
        return bipartite;
    }

    int color(int i) {
        return c[i];
    }

    int find(int i) {
        if (ds[i] == i) return i;
        return ds[i] = find(ds[i]);
    }

    void join(int i, int j) {
        if (find(i) == find(j) && c[i] == c[j]) bipartite = false;
        else if (find(i) != find(j)) {
            bool change = c[i] == c[j];

            i = find(i); j = find(j);
            if (sz[i] > sz[j]) swap(i, j);
            sz[j] += sz[i];
            ds[i] = ds[j];

            for (int v: children[i]) {
                children[j].push_back(v);
                if (change) c[v] = 1 - c[v];
            }
        }
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    cin >> N >> M;

    vector<tuple<int, int, int>> roads(M);
    for (int i = 0; i < M; i++) {
        int x, y, b;
        cin >> x >> y >> b;
        roads[i] = make_tuple(b, x - 1, y - 1);
    }

    sort(roads.rbegin(), roads.rend());

    int l = 0;
    DSU solver(N);
    vector<int> ans;

    while (l < M && solver.is_bipartite()) {
        int r = l + 1;
        while (r < M && get<0>(roads[r]) == get<0>(roads[l])) r++;

        bool possible = false;
        for (int i = l; i < r && i < l + 3 && !possible; i++)
            for (int j = i + 1; j < r && !possible; j++) {
                auto [_, u1, v1] = roads[i];
                auto [__, u2, v2] = roads[j];

                if (u1 == u2 || u1 == v2 || v1 == u2 || v1 == v2) continue;
                if (solver.find(u1) == solver.find(v1) && solver.color(u1)
                    != solver.color(v1)) continue;
                if (solver.find(u2) == solver.find(v2) && solver.color(u2)
                    != solver.color(v2)) continue;

                int c1 = solver.find(u1) == solver.find(v1) ? solver.color(
                    u1) : -1;
                int c2 = solver.find(u2) == solver.find(v2) ? solver.color(
                    u2) : -1;

                if (c1 == -1 && c2 == -1) {
                    if (solver.find(u1) != solver.find(u2) && solver.find(
                        u1) != solver.find(v2)) possible = true;
                    else if (solver.find(v1) != solver.find(u2) && solver.
                        find(v1) != solver.find(v2)) possible = true;
                    else if (solver.find(u1) == solver.find(u2) && solver.
                        color(u1) != solver.color(u2)) possible = true;
                    else if (solver.find(u1) == solver.find(v2) && solver.
                        color(u1) != solver.color(v2)) possible = true;
                }
                else if (c1 == -1 || c2 == -1 || c1 != c2) possible = true;
            }

        if (possible) ans.push_back(get<0>(roads[l]));

        for (int i = l; i < r; i++) {
            auto [_, u, v] = roads[i];
            solver.join(u, v);
        }

        l = r;
    }

    if (solver.is_bipartite()) ans.push_back(0);

    if (ans.empty()) cout << "IMPOSSIBLE\n";
    else {
        reverse(ans.begin(), ans.end());
        for (int w: ans) cout << w << "\n";
    }
}
```

5.5 E

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    queue<int> times[2];
    for (int i = 0; i < N; i++) {
        int t, d;
        cin >> t >> d;
        times[d].push(t);
    }

    int available = INT_MAX, dir = -1;
    for (int i = 0; i < 2; i++)
        if (!times[i].empty() && times[i].front() + 10 < available)
            available = times[i].front() + 10, dir = i;

    times[dir].pop();
```

```
    while (!times[dir].empty() || !times[1 - dir].empty()) {
        if (!times[dir].empty() && (times[dir].front() < available || times[1 - dir].empty() || times[dir].front() < times[1 - dir].front())) {
            available = times[dir].front() + 10;
            times[dir].pop();
        } else {
            dir = 1 - dir;
            int navailable = max(available, times[dir].front()) + 10;
            while (!times[dir].empty() && times[dir].front() < available) times[dir].pop();
            available = navailable;
        }
    }

    cout << available << "\n";
}
```

5.6 G

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const llong MAX = 1e15 + 5;

map<llong, string> memo;

string solve(vector<pair<llong, int>> &divs, llong n) {
    if (n == 1) return "";

    if (memo.count(n))
        return memo[n];

    string ans = "IMPOSSIBLE";
    for (auto [fib, idx]: divs)
        if (!(n % fib)) {
            string suffix = solve(divs, n / fib);
            if (suffix == "IMPOSSIBLE") continue;

            string nans = string(idx - 1, 'A') + "B" + suffix;
            if (ans == "IMPOSSIBLE" || nans < ans) ans = nans;
        }

    return memo[n] = ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    vector<llong> fibs;
    llong a = 1LL, b = 1LL;

    while (a + b < MAX) {
        fibs.push_back(a + b);
        llong aux = b;
        b = a + b;
        a = aux;
    }

    llong N; cin >> N;

    vector<pair<llong, int>> divs;
    for (int i = 0; i < (int) fibs.size(); i++)
        if (!(N % fibs[i]))
            divs.emplace_back(fibs[i], i + 2);

    string ans = solve(divs, N);
    cout << ans << "\n";
}
```

5.7 H

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, K;
    cin >> N >> K;

    vector<vector<int>> numbers(K, vector<int>()), positions(K, vector<int>());
    for (int i = 0; i < N; i++) {
        int n, c;
        cin >> n >> c;

        numbers[c - 1].push_back(n - 1);
        positions[c - 1].push_back(i);
    }

    bool possible = true;
    for (int i = 0; i < K && possible; i++) {
        sort(numbers[i].begin(), numbers[i].end());
        sort(positions[i].begin(), positions[i].end());
        if (numbers[i] != positions[i]) possible = false;
    }

    cout << (possible ? "Y" : "N") << "\n";
}
```

5.8 K

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int T, D, M;
    cin >> T >> D >> M;

    vector<int> y(M + 2, 0);
    for (int i = 1; i <= M; i++)
        cin >> y[i];

    y[M + 1] = D;

    bool possible = false;
    for (int i = 1; i <= M + 1 && !possible; i++)
        if (y[i] - y[i - 1] >= T) possible = true;

    cout << (possible ? "Y" : "N") << "\n";
}
```

5.9 L

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int LOG = 20;
const int MOD = 1e9 + 7;

struct DSU {
private:
    int n;
    vector<int> ds, sz;

public:
    DSU() {}
    DSU(int n) {
        this->n = n;
        sz.assign(n, 1);

        ds.resize(n);
        iota(ds.begin(), ds.end(), 0);
    }

    int find(int i) {
        if (ds[i] == i) return i;
        return ds[i] = find(ds[i]);
    }

    void join(int i, int j) {
        i = find(i); j = find(j);
        if (sz[i] > sz[j]) swap(i, j);
        if (i != j) sz[j] += sz[i];
        ds[i] = j;
    }
};

DSU ds[LOG];

void update(int i, int l1, int r1, int l2, int r2, int N) {
    if (ds[i].find(l1) == ds[i].find(i ? l2 : 2 * N - l2 - 1)) return;
    ds[i].join(l1, i ? l2 : 2 * N - l2 - 1);

    if (i) {
        int m1 = (l1 + r1) / 2;
        int m2 = (l2 + r2) / 2;

        update(i - 1, l1, m1, l2, m2, N);
        update(i - 1, m1 + 1, r1, m2 + 1, r2, N);
    }
}

void update(int l1, int r1, int l2, int r2, int N) {
    int idx = __lg(r1 - l1 + 1);
    int nl1 = l1 + (1 << idx);
    int nl2 = l2 + (1 << idx);

    update(idx, l1, nl1 - 1, l2, nl2 - 1, N);
    if (nl1 < r1) update(nl1, r1, nl2, r2, N);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    cin >> N >> M;

    string s; cin >> s;

    ds[0] = DSU(N);
    for (int i = 1; i < LOG; i++)
        ds[i] = DSU(2 * N);

    for (int i = 0; i < M; i++) {
        int l, r;
        cin >> l >> r;
        update(l - 1, r - 1, 2 * N - r, 2 * N - l, N);
    }

    vector<vector<int>> groups(N, vector<int>());
    for (int i = 0; i < N; i++)
        groups[ds[0].find(i)].push_back(i);

    int ans = 1;
    for (int i = 0; i < N; i++) {
        if (groups[i].empty()) continue;

        bool zero = false, one = false, unknown = false;
        for (int j: groups[i]) {
            if (s[j] == '0') zero = true;
            else if (s[j] == '1') one = true;
            else unknown = true;
        }

        if (zero && one) ans = 0;
        else if (!zero && !one) ans = (2LL * ans) % MOD;
    }

    cout << ans << "\n";
}
```

5.10 M

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 1e5 + 5;

struct FenwickTree {
private:
    int n;
    vector<int> tree;

public:
    FenwickTree(int n) {
        this->n = n + 1;
        tree.assign(n + 1, 0);
    }

    void add(int i, int v) {
        if (!i) tree[i] += v;
        else {
            while (i < n) {
                tree[i] += v;
                i += LSO(i);
            }
        }
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

private:
```

```
    int LSO(int i) {
        return i & -i;
    }

    int sum(int i) {
        if (i < 0) return 0;

        int ans = tree[0];
        while (i) {
            ans += tree[i];
            i -= LSO(i);
        }

        return ans;
    }
};

vector<int> tree[MAX];
vector<int> pos, monarch;

void init(int &vis, int node = 0) {
    pos[node] = vis;
    monarch[vis++] = node;

    for (int child: tree[node])
        init(vis, child);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int Q; cin >> Q;

    int id = 1;
    vector<int> queries;

    while (Q--) {
        int t, x;
        cin >> t >> x;

        if (t == 1) tree[x - 1].push_back(id++);
        else queries.push_back(x - 1);
    }

    int vis = 0;
    pos.resize(id);
    monarch.resize(id);
    init(vis);

    FenwickTree solver(id);
    for (int x: queries) {
        solver.add(pos[x], 1);

        int l = 0, r = id - 1, ans = -1;
        while (l <= r) {
            int mid = (l + r) / 2;

            if (solver.sum(0, mid) >= mid + 1) l = mid + 1;
            else r = mid - 1, ans = monarch[mid] + 1;
        }

        assert(ans != -1);
        cout << ans << "\n";
    }
}
```

5.11 N

```

#include <bits/stdc++.h>

using namespace std;

#define llong long long int

struct MergeSortTree {
private:
    int n;
    vector<int> a;
    vector<vector<int>> tree;

public:
    MergeSortTree(int n, vector<int> &a) {
        this->n = n;
        this->a = a;

        tree.resize(4 * n);
        build(1, 1, n);
    }

    int query(int l, int r, int v) {
        return query(1, 1, n, l, r, v);
    }

private:
    void merge(vector<int> &a, vector<int> &b, vector<int> &c) {
        int i = 0, j = 0, k = 0;
        int nb = b.size(), nc = c.size();

        a.resize(nb + nc);
        while (j < nb || k < nc) {
            if (j >= nb) a[i++] = c[k++];
            else if (k >= nc) a[i++] = b[j++];
            else if (b[j] <= c[k]) a[i++] = b[j++];
            else a[i++] = c[k++];
        }
    }

    void build(int node, int l, int r) {
        if (l == r) tree[node] = vector<int>(1, a[l]);
        else {
            int m = (l + r) / 2;
            build(2 * node, l, m);
            build(2 * node + 1, m + 1, r);
            merge(tree[node], tree[2 * node], tree[2 * node + 1]);
        }
    }

    int query(int node, int l, int r, int ql, int qr, int v) {
        if (r < ql || l > qr) return 0;
        if (ql <= l && r <= qr) {
            auto it = lower_bound(tree[node].begin(), tree[node].end(), v);
            return tree[node].end() - it;
        }

        int m = (l + r) / 2;
        return query(2 * node, l, m, ql, qr, v) + query(2 * node + 1, m +
            1, r, ql, qr, v);
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int Y, N;
    cin >> Y >> N;

    vector<int> x(Y + 1);
    for (int i = 1; i <= Y; i++)
        cin >> x[i];

```

```
    MergeSortTree tree(Y, x);
    while (N--) {
        int a, p, f;
        cin >> a >> p >> f;

        if (p <= x[a] || !f) cout << "0\n";
        else {
            int ans = tree.query(a + 1, a + f, p);
            cout << ans << "\n";
        }
    }
}
```

6 2022

6.1 A

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;
    string s; cin >> s;

    int idx = 0, ans = 0;
    while (idx < N) {
        while (idx < N && s[idx] == 'b') idx++;

        int cnt = 0;
        while (idx < N && s[idx] == 'a')
            idx++, cnt++;

        if (cnt > 1) ans += cnt;
    }

    cout << ans << "\n";
}
```

6.2 B

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 30;
const int MOD = 1e9 + 7;

int memo[2][8][MAX];

int win(int a, int b, int c, bool pos = false, int tight = 7, int i = MAX - 1) {
    if (i < 0) return pos;

    int &ans = memo[pos][tight][i];
    if (~ans) return ans;

    ans = 0;
    for (int digits = 0; digits < 8; digits++) {
        if ((tight & 1) && (digits & 1) && !(a & (1 << i))) continue;
        if ((tight & 2) && (digits & 2) && !(b & (1 << i))) continue;
        if ((tight & 4) && (digits & 4) && !(c & (1 << i))) continue;

        bool atight = (tight & 1) && !!(digits & 1) == !!(a & (1 << i));
        bool btight = (tight & 2) && !!(digits & 2) == !!(b & (1 << i));
        bool ctight = (tight & 4) && !!(digits & 4) == !!(c & (1 << i));

        bool npos = pos || (__builtin_popcount(digits) & 1);
        int ntight = atight | (btight << 1) | (ctight << 2);

        ans = (ans + win(a, b, c, npos, ntight, i - 1)) % MOD;
    }

    return ans;
}

int exp(int a, int n) {
    if (!n) return 1;

    int ans = exp(a, n >> 1);
    ans = (1LL * ans * ans) % MOD;

    if (!(n & 1)) return ans;
    return (1LL * ans * a) % MOD;
}

int mod_mult_inv(int a, int m) {
    return exp(a, m - 2);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int L[3], R[3];
    for (int i = 0; i < 3; i++)
        cin >> L[i] >> R[i];

    int ans = 0;
    for (int i = 0; i < 8; i++) {
        memset(memo, -1, sizeof memo);

        int a = i & 1 ? L[0] - 1 : R[0];
        int b = i & 2 ? L[1] - 1 : R[1];
        int c = i & 4 ? L[2] - 1 : R[2];

        int curr = win(a, b, c);
        if (__builtin_popcount(i) & 1) curr = (MOD - curr) % MOD;
        ans = (ans + curr) % MOD;
    }

    int total = R[0] - L[0] + 1;
    total = (total * (R[1] - L[1] + 1LL)) % MOD;
    total = (total * (R[2] - L[2] + 1LL)) % MOD;

    ans = (1LL * ans * mod_mult_inv(total, MOD)) % MOD;
    cout << ans << "\n";
}
```

6.3 C

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 1e3;
const int INF = 1e9 + 5;
```

```
const int dr[4] = { 1, 0, -1, 0 };
const int dc[4] = { 0, 1, 0, -1 };

bool vis[MAX + 1][MAX + 1];
bool blocked[MAX + 1][MAX + 1][4];

int size(int row, int col) {
    if (row < 0 || row > MAX || col < 0 || col > MAX) return -INF;
    if (vis[row][col]) return 0;

    int ans = 1;
    vis[row][col] = true;

    for (int i = 0; i < 4; i++)
        if (!blocked[row][col][i])
            ans += size(row + dr[i], col + dc[i]);

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    vector<int> X(N + 1), Y(N + 1);
    for (int i = 0; i <= N; i++)
        cin >> X[i] >> Y[i];

    memset(blocked, 0, sizeof blocked);
    for (int i = 1; i <= N; i++) {
        if (X[i] == X[i - 1]) {
            for (int j = min(Y[i], Y[i - 1]); j < max(Y[i], Y[i - 1]); j++)
                blocked[j][X[i] - 1][1] = blocked[j][X[i]][3] = true;
        } else {
            assert(Y[i] == Y[i - 1]);
            for (int j = min(X[i], X[i - 1]); j < max(X[i], X[i - 1]); j++)
                blocked[Y[i] - 1][j][0] = blocked[Y[i]][j][2] = true;
        }
    }

    int ans = 0;
    memset(vis, 0, sizeof vis);

    for (int i = 0; i <= MAX; i++)
        for (int j = 0; j <= MAX; j++)
            ans = max(ans, size(i, j));

    cout << ans << "\n";
}
```

6.4 D

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int N;

int dist(int x, int y) {
    int ans = 0;
    while (x != (1 << (N - 1)) || y != (1 << (N - 1))) {
        ans++;

        if (x & (1 << (N - 1))) x ^= (1 << (N - 1));
        x <<= 1;

        if (y & (1 << (N - 1))) y ^= (1 << (N - 1));
        y <<= 1;
    }

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> N;

    int x, y;
    cin >> x >> y;

    int ans = dist(x, y);
    cout << ans << "\n";
}
```

6.5 E

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 1e6 + 5;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    int cnt[MAX] = {0}, ans = 0;
    for (int i = 0; i < N; i++) {
        int H; cin >> H;

        cnt[H]++;
        if (!cnt[H + 1]) ans++;
        else cnt[H + 1]--;
    }

    cout << ans << "\n";
}
```

6.6 F

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, C;
    cin >> N >> C;

    int matches = 0;
    string ans = "";
    map<string, int> cnt;

    for (int i = 0; i < N; i++) {
```

```
        string S; cin >> S;

        int idx = 0;
        while (idx < C && S[idx] != '*') idx++;
        assert(idx < C);

        for (char c = 'a'; c <= 'z'; c++) {
            S[idx] = c;
            cnt[S]++;

            if (cnt[S] > matches || (cnt[S] == matches && S < ans))
                matches = cnt[S], ans = S;
        }
    }

    cout << ans << " " << matches << "\n";
}
```

6.7 G

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int
#define Vector Point

template<typename T>
struct Point {
    T x, y;

    Point(T x = 0, T y = 0): x(x), y(y) {}

    Vector operator -(Point<T> other) {
        return Vector<T>(x - other.x, y - other.y);
    }

    T operator /(Vector<T> other) {
        return x * other.y - y * other.x;
    }

    bool operator ==(Point<T> const &other) const {
        return x == other.x && y == other.y;
    }

    bool operator <(Point<T> const &other) const {
        if (x == other.x)
            return y < other.y;

        return x < other.x;
    }
};

template<typename T>
struct Segment {
    Point<T> P, Q;

    Segment(Point<T> P = Point<T>(), Point<T> Q = Point<T>()) {
        if (Q < P) swap(P, Q);
        this->P = P;
        this->Q = Q;
    }

    bool operator <(Segment<T> const &other) const {
        if (P == other.P)
            return Q < other.Q;

        return P < other.P;
    }
};

vector<llong> darea;
vector<vector<int>> tree;
vector<vector<llong>> memo;

llong min_area(bool can_remove = true, int node = 0, int parent = -1) {
    llong &ans = memo[can_remove][node];
    if (~ans) return ans;

    llong keep = darea[node], remove = 0LL;
    for (int child: tree[node])
        if (child != parent) {
            keep += min_area(true, child, node);
            remove += min_area(false, child, node);
        }

    ans = keep;
    if (can_remove && (int) tree[node].size() == 3)
        ans = min(ans, remove);

    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    darea.resize(N);
    tree.assign(N, vector<int>());
    map<Segment<llong>, int> triangle;

    for (int i = 0; i < N; i++) {
        int x1, y1, x2, y2, x3, y3;
        cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;

        Point<llong> A(x1, y1), B(x2, y2), C(x3, y3);
        Segment<llong> AB(A, B), AC(A, C), BC(B, C);

        if (triangle.count(AB)) {
            int v = triangle[AB];
            tree[i].push_back(v);
            tree[v].push_back(i);
        }

        if (triangle.count(AC)) {
            int v = triangle[AC];
            tree[i].push_back(v);
            tree[v].push_back(i);
        }

        if (triangle.count(BC)) {
            int v = triangle[BC];
            tree[i].push_back(v);
            tree[v].push_back(i);
        }

        darea[i] = abs((B - A) / (C - A));
        triangle[AB] = triangle[AC] = triangle[BC] = i;
    }

    memo.assign(2, vector<llong>(N + 5, -1LL));

    llong ans = min_area();
    cout << (ans / 2LL) << "." << (ans & 1LL ? "5" : "0") << "\n";
}
```

6.8 H

```
#include <bits/stdc++.h>

```

```cpp
using namespace std;

#define llong long long int

struct SCC {
private:
    int n;
    stack<int> order;
    vector<bool> vis;
    vector<int> component;
    vector<vector<int>> g, gt;

public:
    SCC(int n, vector<vector<int>> &g) {
        this->n = n;
        this->g = g;

        gt.assign(n, vector<int>());
        for (int i = 0; i < n; i++)
            for (auto v: g[i])
                gt[v].push_back(i);
    }

    vector<int> get_components() {
        vis.assign(n, false);
        for (int i = 0; i < n; i++)
            if (!vis[i]) toposort(i);

        int idx = 0;
        component.assign(n, -1);

        while (!order.empty()) {
            int v = order.top(); order.pop();
            if (component[v] == -1)
                get_component(v, idx++);
        }

        return component;
    }

private:
    void toposort(int v) {
        if (vis[v]) return;
        vis[v] = true;

        for (auto u: g[v])
            toposort(u);

        order.push(v);
    }

    void get_component(int v, int c) {
        if (~component[v]) return;
        component[v] = c;

        for (auto u: gt[v])
            get_component(u, c);
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    cin >> N >> M;

    vector<vector<int>> g(N, vector<int>());
    for (int i = 0; i < M; i++) {
        int R, S;
        cin >> R >> S;
        g[R - 1].emplace_back(S - 1);
    }

    vector<int> component = SCC(N, g).get_components();
    int num_components = *max_element(component.begin(), component.end()) +
        1;

    if (num_components == 1) cout << "0\n";
    else {
        vector<int> in(num_components, 0), out(num_components, 0);
        for (int i = 0; i < N; i++)
            for (int v: g[i])
                if (component[i] != component[v]) {
                    in[component[v]]++;
                    out[component[i]]++;
                }

        int need_in = 0, need_out = 0;
        for (int i = 0; i < num_components; i++) {
            if (!in[i]) need_in++;
            if (!out[i]) need_out++;
        }

        int ans = max(need_in, need_out);
        cout << ans << "\n";
    }
}
```

6.9 I

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    bool successful = true;
    for (int i = 0; i < 8 && successful; i++) {
        int N; cin >> N;
        if (N == 9)
            successful = false;
    }

    cout << (successful ? "S" : "F") << "\n";
}
```

6.10 J

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 13;

int points(int card) {
    if (card <= 10)
        return card;

    return 10;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
```

```cpp

    int N; cin >> N;

    int cnt[MAX] = {0}, john = 0, mary = 0;
    for (int i = 0; i < 2; i++) {
        int c; cin >> c;

        cnt[c - 1]++;
        john += points(c);
    }

    for (int i = 0; i < 2; i++) {
        int c; cin >> c;

        cnt[c - 1]++;
        mary += points(c);
    }

    for (int i = 0; i < N; i++) {
        int c; cin >> c;

        cnt[c - 1]++;
        john += points(c);
        mary += points(c);
    }

    int ans = -1;
    for (int i = 0; i < MAX && mary + points(i + 1) <= 23 && ans == -1; i
        ++)
        if (cnt[i] < 4 && (john + points(i + 1) > 23 || mary + points(i +
            1) == 23))
            ans = i + 1;

    cout << ans << "\n";
}
```

6.11 K

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 10;
const int MOD = 1e9;

struct Polynomial {
    int deg;
    vector<int> p;

    Polynomial(int deg = 0) {
        this->deg = deg;
        p.assign(deg + 1, 0);
    }

    int& operator [](int index) {
        return p[index];
    }

    Polynomial operator +(Polynomial other) {
        Polynomial ans(max(deg, other.deg));
        for (int i = 0; i <= min(deg, other.deg); i++)
            ans[i] = (p[i] + other[i]) % MOD;

        for (int i = min(deg, other.deg) + 1; i <= max(deg, other.deg); i
            ++)
            ans[i] = i > deg ? other[i] : p[i];

        return ans;
    }

    Polynomial operator *(Polynomial other) {
        Polynomial ans(max(deg, other.deg));
        for (int i = 0; i <= deg; i++)
            for (int j = 0; j <= other.deg && i + j <= max(deg, other.deg);
                j++)
                ans[i + j] = (ans[i + j] + 1LL * p[i] * other[j]) % MOD;

        return ans;
    }
};

template<typename T>
struct Matrix {
    int rows, cols;
    T m[MAX][MAX];

    Matrix(int rows, int cols): rows(rows), cols(cols) {
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                m[i][j] = T();
    }

    Matrix<T> operator *(Matrix<T> &other) {
        Matrix<T> ans(rows, other.cols);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < other.cols; j++)
                for (int k = 0; k < cols; k++)
                    ans[i][j] = ans[i][j] + m[i][k] * other[k][j];

        return ans;
    }

    T* operator [](int row) {
        return m[row];
    }
};

template<typename T>
Matrix<T> exp(Matrix<T> &A, int n) {
    if (n == 1) return A;

    Matrix<T> ans = exp(A, n >> 1);
    ans = ans * ans;

    if (!(n & 1)) return ans;
    return ans * A;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M, K;
    cin >> N >> M >> K;

    vector<Polynomial> jumps(10, Polynomial(K));
    for (int i = 0; i < M; i++) {
        int d, p;
        cin >> d >> p;
        jumps[d - 1][p]++;
    }

    if (N == 1) cout << "1\n";
    else {
        Matrix<Polynomial> A(10, 10);
        for (int i = 0; i < 10; i++)
            A[0][i] = jumps[i];

        for (int i = 1; i < 10; i++)
            for (int j = 0; j < 10; j++) {
```

```
                A[i][j] = Polynomial();
                A[i][j][0] = (j + 1 == i);
            }

        A = exp(A, N - 1);

        int ans = 0;
        for (int i = 0; i <= K; i++)
            ans = (ans + A[0][0][i]) % MOD;

        cout << ans << "\n";
    }
}
```

6.12 L

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

llong sum = 0LL;
vector<vector<pair<int, int>>> tree;
vector<int> C, sz, total, cnt;
vector<llong> ans;

void init(int node = 0, int parent = -1) {
    sz[node] = 1;
    total[C[node]]++;

    for (auto [child, _]: tree[node])
        if (child != parent) {
            init(child, node);
            sz[node] += sz[child];
        }
}

void add(int node, int parent, int big, int v) {
    int color = C[node];
    sum -= 1LL * cnt[color] * (total[color] - cnt[color]);

    cnt[color] += v;
    sum += 1LL * cnt[color] * (total[color] - cnt[color]);

    for (auto [child, _]: tree[node])
        if (child != parent && child != big)
            add(child, node, big, v);
}

void solve(int node = 0, int parent = -1, int edge = 0, bool keep = true) {
    int big = -1, bedge = -1, mx = INT_MIN;
    for (auto [child, id]: tree[node])
        if (child != parent && sz[child] > mx) {
            bedge = id;
            big = child;
            mx = sz[child];
        }

    for (auto [child, id]: tree[node])
        if (child != parent && child != big)
            solve(child, node, id, false);

    if (big != -1) solve(big, node, bedge);

    add(node, parent, big, 1);
    ans[edge] = sum;

    if (!keep) add(node, parent, -1, -1);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    C.resize(N);
    for (int i = 0; i < N; i++)
        cin >> C[i];

    tree.assign(N, vector<pair<int, int>>());
    for (int i = 1; i < N; i++) {
        int u, v;
        cin >> u >> v;

        tree[u - 1].emplace_back(v - 1, i);
        tree[v - 1].emplace_back(u - 1, i);
    }

    sz.resize(N);
    total.assign(N + 1, 0);
    init();

    cnt.assign(N + 1, 0);
    ans.assign(N, 0LL);
    solve();

    for (int i = 1; i < N; i++) {
        if (i > 1) cout << " ";
        cout << ans[i];
    }

    cout << "\n";
}
```

6.13 M

```
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

const int MAX = 1e5 + 5;

int last_prime[MAX] = {0};
vector<int> primes[MAX];

void get_primes(int x) {
    vector<int> &ans = primes[x];
    if (!ans.empty()) return;

    while (x > 1) {
        int prime = last_prime[x];
        while (x > 1 && last_prime[x] == prime)
            x /= prime;

        ans.push_back(prime);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    for (int i = 2; i < MAX; i++)
        if (!last_prime[i])
            for (int j = i; j < MAX; j += i)
                last_prime[j] = i;
```

```cpp

    int N, Q;
    cin >> N >> Q;

    vector<int> A(N);
    for (int i = 0; i < N; i++)
        cin >> A[i];

    vector<int> c(Q), d(Q);
    for (int i = 0; i < Q; i++)
        cin >> c[i] >> d[i];

    int max_size = N;
    llong sums[MAX] = {0LL};
    vector<int> l(N, 0), r(N, Q - 1), ans(N, -1);

    while (max_size > 1) {
        max_size = 0;
        vector<vector<int>> groups(Q, vector<int>());

        for (int i = 0; i < N; i++)
            if (l[i] <= r[i]) {
                int mid = (l[i] + r[i]) / 2;
                groups[mid].push_back(i + 1);

                int sz = groups[mid].size();
                max_size = max(max_size, sz);
            }

        memset(sums, 0, sizeof sums);
        for (int i = 0; i < Q; i++) {
            get_primes(c[i]);
            int num_primes = primes[c[i]].size();

            for (int j = 1; j < (1 << num_primes); j++) {
                int num = 1;
                for (int k = 0; k < num_primes; k++)
                    if (j & (1 << k))
                        num *= primes[c[i]][k];

                sums[num] += (__builtin_popcount(j) & 1 ? d[i] : -d[i]);
            }

            for (int id: groups[i]) {
                get_primes(id);
                num_primes = primes[id].size();

                llong sum = 0LL;
                for (int j = 1; j < (1 << num_primes); j++) {
                    int num = 1;
                    for (int k = 0; k < num_primes; k++)
                        if (j & (1 << k))
                            num *= primes[id][k];

                    sum += sums[num];
                }

                if (sum < A[id - 1]) l[id - 1] = i + 1;
                else r[id - 1] = i - 1, ans[id - 1] = i + 1;
            }
        }
    }

    for (int i = 0; i < N; i++)
        cout << ans[i] << "\n";
}
```

6.14 N

```cpp
#include <bits/stdc++.h>

using namespace std;

#define llong long long int

struct SegmentTree {
private:
    int n;
    vector<int> cnt;
    vector<llong> sum;

public:
    SegmentTree(int n) {
        this->n = n;
        cnt.assign(4 * n, 0);
        sum.assign(4 * n, 0LL);
    }

    void update(int i, int c, int v) {
        update(1, 1, n, i, c, v);
    }

    llong query(int k) {
        return query(1, 1, n, k);
    }

private:
    void update(int node, int l, int r, int i, int c, int v) {
        if (l == r) {
            cnt[node] += c;
            sum[node] += v;
        } else {
            int m = (l + r) / 2;
            if (i <= m) update(2 * node, l, m, i, c, v);
            else update(2 * node + 1, m + 1, r, i, c, v);

            cnt[node] = cnt[2 * node] + cnt[2 * node + 1];
            sum[node] = sum[2 * node] + sum[2 * node + 1];
        }
    }

    llong query(int node, int l, int r, int k) {
        if (l == r) {
            assert(cnt[node] >= k);
            return k * sum[node] / cnt[node];
        }

        int m = (l + r) / 2;
        if (cnt[2 * node + 1] >= k) return query(2 * node + 1, m + 1, r, k);
        return sum[2 * node + 1] + query(2 * node, l, m, k - cnt[2 * node + 1]);
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N; cin >> N;

    vector<int> A(N);
    for (int i = 0; i < N; i++)
        cin >> A[i];

    vector<int> B(N);
    for (int i = 0; i < N; i++)
        cin >> B[i];

    int K, L;
    cin >> K >> L;

    vector<int> order = B;
```

```
    sort(order.begin(), order.end());
    order.erase(unique(order.begin(), order.end()), order.end());
    int sz = order.size();

    llong fsum = 0LL;
    SegmentTree tree(sz);

    for (int i = 0; i < K; i++) {
        fsum += A[i];

        int idx = lower_bound(order.begin(), order.end(), B[i]) - order.begin();
        tree.update(idx + 1, 1, B[i]);
    }

    llong ans = fsum + tree.query(L);
    for (int i = 1; i <= K; i++) {
        fsum -= A[K - i];
        fsum += A[N - i];

        int idx = lower_bound(order.begin(), order.end(), B[K - i]) - order.begin();
        tree.update(idx + 1, -1, -B[K - i]);

        idx = lower_bound(order.begin(), order.end(), B[N - i]) - order.begin();
        tree.update(idx + 1, 1, B[N - i]);

        ans = max(ans, fsum + tree.query(L));
    }

    cout << ans << "\n";
}
```